

Project Proposal

RIMIT Education B2B Aggregator System

Document Information	Details
Proposal Version	1.0
Date	July 1, 2026
Prepared By	Bytestrone
Document Status	Final Board Submission

Table of Contents

- 1. Executive Summary
- 2. Project Objectives
- 3. Proposed Solution
- 4. Solution Architecture
- 5. Technology Stack
- 6. Project Governance
- 7. Project Delivery Methodology
- 8. Implementation Timeline
- 9. Support and Maintenance
- 10. Risk Management
- 11. Assumptions
- 12. Deliverables
- 13. Conclusion

1. Executive Summary

This proposal outlines the design, development, deployment, and operational strategy for the RIMIT Education B2B Aggregator System.

The proposed solution follows a minimalist architecture that focuses on simplicity, performance, security, and maintainability. Instead of introducing unnecessary infrastructure complexity, the platform leverages mature technologies such as Django and PostgreSQL to provide enterprise-grade performance while reducing operational costs.

The architecture emphasizes:

- Reduced Total Cost of Ownership (TCO)
- Faster development cycles
- Secure multi-tenant architecture
- High availability
- Simplified deployment
- Long-term maintainability

The project will follow a structured discovery process followed by a locked blueprint methodology to ensure predictable delivery timelines and controlled project scope.

2. Project Objectives

The primary objectives of the proposed system are:

- Build a centralized B2B education aggregation platform.
- Support secure multi-tenancy for multiple education partners.
- Provide efficient course and university search capabilities.
- Deliver a scalable administration platform.
- Enable partner onboarding with minimal operational effort.
- Maintain low infrastructure and maintenance costs.
- Ensure predictable project delivery through controlled governance.

3. Proposed Solution

The proposed solution adopts a minimalist engineering philosophy by relying on proven platform capabilities instead of introducing multiple external services.

Key characteristics include:

- Monolithic backend architecture
- Database-native search
- Secure row-level multi-tenancy

- Background job processing
- Reverse proxy based traffic management
- Simplified deployment pipeline

This approach minimizes infrastructure complexity while maintaining enterprise-level reliability and performance.

4. Solution Architecture

4.1 Infrastructure Layer

Reverse Proxy

- Nginx
- SSL/TLS termination
- Request routing
- Compression
- Rate limiting
- Basic DDoS mitigation

4.2 Application Layer

Backend Framework

- Django (Python)

Responsibilities:

- Business logic
- Authentication
- Authorization
- REST APIs
- Admin Portal
- Workflow management

4.3 Database Layer

PostgreSQL

Responsibilities:

- Primary data storage
- Full-text search
- Transaction management
- Row-Level Security (RLS)
- Data integrity

4.4 Search Engine

Instead of external search platforms such as Elasticsearch or OpenSearch, the system utilizes PostgreSQL native capabilities.

Features include:

- tsvector
- GIN indexes
- Native ranking
- Full-text search
- Lower operational cost
- Reduced infrastructure footprint

4.5 Multi-Tenant Security

Tenant isolation is enforced using PostgreSQL Row-Level Security (RLS).

Benefits include:

- Database-level isolation
- Secure tenant separation
- Reduced application complexity
- Strong data governance

4.6 Background Processing

Background operations are managed using:

- Celery
- Redis

Typical background jobs include:

- Document processing
- Email notifications
- WhatsApp notifications
- Scheduled jobs
- Data synchronization

5. Technology Stack

Layer	Technology
Reverse Proxy	Nginx
Backend	Django (Python)
Database	PostgreSQL
Search	PostgreSQL Full Text Search
Cache	Redis
Background Jobs	Celery
Containerization	Docker
Source Control	Azure DevOps
Project Management	Azure DevOps
Deployment	Linux

6. Project Governance

6.1 Discovery Phase

Duration:

3 Weeks

Activities:

- Business requirement gathering
- Functional analysis
- Database design
- API specifications
- UI wireframes
- Architecture validation

Outputs:

- Approved database schema
- Approved API contract
- Approved wireframes
- Functional specification
- Technical specification

The approved documents become the project's single source of truth.

6.2 Locked Blueprint Methodology

After completion of the Discovery Phase:

- Functional scope is frozen.
- Database schema is locked.
- API contracts are locked.
- UI layouts are locked.

Subsequent changes require a formal Change Request (CR).

6.3 UI Revision Policy

To ensure project schedule stability:

- Maximum two revision rounds per module
- Additional revisions require Change Request approval

6.4 Change Management

All project approvals shall be maintained within Azure DevOps.

No project changes will be accepted through:

- Email
- Phone calls
- Informal meetings
- Messaging platforms

Scope changes require:

- Change Request
- Cost estimation
- Timeline approval
- Client sign-off

6.5 Third-Party Integration Strategy

External integrations shall be validated using Docker-based mock services.

Examples include:

- Meta APIs
- WhatsApp APIs
- External webhooks

Project milestones will be measured using sandbox validation rather than production approval timelines.

7. Project Delivery Methodology

The implementation will follow four structured phases.

Phase 1

Discovery

Duration: 3 Weeks

Activities:

- Requirement gathering
- Database schema design
- API definition
- UI wireframes

- Approval process

Phase 2

Alpha Development

Duration: 5 Weeks

Activities:

- Aggregator Hub development
- Rules Engine
- Authentication
- PostgreSQL RLS implementation
- Administration portal

Phase 3

Beta Development

Duration: 4 Weeks

Activities:

- Sub-center portal
- Appsmith integration
- Lead ingestion
- Notification services
- Integration testing

Phase 4

UAT and Final Validation

Duration: 4 Weeks

Activities:

- User Acceptance Testing

- Load testing
- Stress testing
- Security verification
- RLS audit
- Documentation
- Deployment preparation

8. Implementation Timeline

Phase	Duration	Activities
Discovery	3 Weeks	Requirements, Database Design, API Contracts, UI Wireframes
Alpha Development	5 Weeks	Core Platform Development, Rules Engine, Admin Portal, RLS
Beta Development	4 Weeks	Partner Portals, Lead Pipeline, Notifications
UAT & Final Testing	4 Weeks	Testing, Documentation, Security Audit, Deployment Readiness

Total Duration: 16 Weeks

9. Support and Maintenance

Operational Stabilization

Duration:

First Month After Go-Live

Includes:

- Production monitoring
- Partner onboarding support
- Critical bug fixes
- Minor workflow improvements
- UI label adjustments

- Operational assistance

Annual Maintenance Contract (AMC)

Following stabilization, the system transitions into an AMC model.

Coverage includes:

- Security updates
- Bug fixes
- Performance monitoring
- Preventive maintenance
- Stability improvements

Service Window

- 9:00 AM to 5:00 PM
- Business Days

Scope

Covered:

- Verified defects
- Security patches
- Maintenance activities

Excluded:

- New modules
- Feature enhancements
- Workflow redesign
- Business process changes

Additional work outside AMC scope shall be billed separately.

10. Risk Management

Risk	Impact	Mitigation
Scope Expansion	High	Standard rule templates only. Custom logic requires Change Request.
Excessive UI Revisions	High	Discovery limited to three weeks with two revision cycles per module.
Third-Party Vendor Delays	High	Sandbox validation used for project milestones.
Cloud Cost Variations	Moderate	Infrastructure costs managed directly through RIMIT cloud account.

11. Assumptions

The proposal assumes:

- Timely client approvals.
- Availability of business stakeholders.
- Access to required third-party sandbox environments.
- Infrastructure provisioned by the client.
- Cloud subscription owned by RIMIT.
- Necessary API credentials provided before implementation.

12. Deliverables

The project includes delivery of:

- Requirement Specification
- Solution Architecture
- Database Design
- API Documentation
- UI Wireframes
- Backend Application
- Administration Portal
- Partner Portal
- Notification Services
- User Documentation

- Technical Documentation
- Deployment Documentation
- UAT Support
- Production Deployment Assistance

13. Conclusion

The proposed RIMIT Education B2B Aggregator System provides a secure, scalable, and cost-effective platform built on a minimalist architecture. By leveraging Django, PostgreSQL, Row-Level Security, and database-native search, the solution minimizes infrastructure complexity while delivering enterprise-grade functionality.

The structured discovery process, locked blueprint methodology, and controlled change management approach provide predictable project execution, reduced delivery risk, and long-term operational stability. This architecture enables RIMIT to efficiently onboard partners, manage education data securely, and scale the platform while maintaining low operational overhead and a sustainable total cost of ownership.